



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Rodríguez Zamora Joshua

N° de Cuenta: 320229594

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE: 2026-1

FECHA DE ENTREGA LÍMITE: 07/09/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Generar una pirámide rubik (pyraminx) de 9 pirámides por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña) Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras

La actividad pidió utilizar como base de todas las figuras, pirámides triangulares. Lo cual fue posible gracias a una pirámide negra que actúa como base a la cual se le añadieron en cada cara 9 pirámides en un determinado color a forma de “stickers”. Primero se modificaron las dimensiones y la definición de la figura para que sea similar a la del pyraminx como se ve a continuación:

```
85 // Pirámide triangular regular
86 void CrearPiramideTriangular()
87 {
88     unsigned int indices_piramide_triangular[] = {
89         0,1,2,
90         1,3,2,
91         3,0,2,
92         1,0,3
93     };
94     GLfloat vertices_piramide_triangular[] = {
95         -0.5f, -0.5f, -0.433f, //0
96         0.5f, -0.5f, -0.433f, //1
97         0.0f, 0.316f, -0.15f, //2
98         0.0f, -0.5f, 0.433f, //3
99     };
100     Mesh* obj1 = new Mesh();
101     obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
102     meshList.push_back(obj1);
103 }
104
105
106
```

Ilustración 1. Definición triángulo pyraminx.

La pirámide base, es asignada con un color negro y la definimos en un origen de $x = 0$, $y = 0$ y $z = -3$. Por lo que la figura aparecerá en este sentido al aparecer. Tomando una perspectiva ortogonal que permitirá visualizar a la figura y rotarla con respecto a sus ejes después con nuevas coordenadas que parten de este origen. Esto permitirá saber como colocar las coordenadas o el sentido de las rotaciones necesarias para la correcta representación de la figura.

```

352
353 //PIRAMIDE NEGRA
354 model = glm::mat4(1.0);
355 //Traslación inicial para posicionar en -Z a los objetos
356 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
357 //otras transformaciones para el objeto
358 model = glm::scale(model, glm::vec3(2.0f, 2.0f, 2.0f));
359 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
360 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
361 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
362 //Traslación para colocar a los objetos en su respectivo lugar
363 model = glm::translate(model, glm::vec3(0.0f, 0.453f, 0.383f));
364 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
365 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
366 //se programe cambio entre proyección ortogonal y perspectiva
367 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
368 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
369 color = glm::vec3(0.0f, 0.0f, 0.0f);
370 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
371 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular

```

Ilustración 2. Definición de base pyraminx.

Una vez que tenemos la base, empezamos a colocar las pirámides de forma que queden representando stickers de color. Para dicha tarea es necesario comprender los ejes y el sentido que tienen x, y y z de modo que se demuestren apenas visibles y limitados para que la figura tome sentido. En general daremos una escala de 0.4 a cada eje de la figura. Los códigos individuales son:

Cara Roja:

```

372
373 ///////////////////////////////////////////////////
374 //STICKERS ROJOS//
375 ///////////////////////////////////////////////////
376
377 //PICO ROJO ARRIBA
378 model = glm::mat4(1.0);
379 //Traslación inicial para posicionar en -Z a los objetos
380 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
381 //otras transformaciones para el objeto
382
383 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
384 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
385 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
386 // Se establece su posición específica
387 model = glm::translate(model, glm::vec3(0.0f, 1.25f, 0.48f));
388 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
389 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
390 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
391 //se programe cambio entre proyección ortogonal y perspectiva
392 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
393 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
394 color = glm::vec3(1.0f, 0.0f, 0.0f);
395 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
396 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular

```

Ilustración 3. Figura roja 1

```

397
398 //MEDIO IZQUIERDA
399 model = glm::mat4(1.0);
400 //Traslación inicial para posicionar en -Z a los objetos
401 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
402 //otras transformaciones para el objeto
403 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
404 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
405 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
406 model = glm::translate(model, glm::vec3(0.28f, 0.76f, 0.32f));
407 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
408 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
409 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
410 //se programe cambio entre proyección ortogonal y perspectiva
411 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
412 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
413 color = glm::vec3(1.0f, 0.0f, 0.0f);
414 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
415 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
416

```

Ilustración 4. Figura roja2

```

416
417 //MEDIO MEDIO INVERSA
418 model = glm::mat4(1.0);
419 //Traslación inicial para posicionar en -Z a los objetos
420 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
421 //otras transformaciones para el objeto
422 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
423 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
424 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
425 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
426 model = glm::rotate(model, -34 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
427 //model = glm::rotate(model, 10 * toRadians, glm::vec3(0.0f, 0.0f, 0.0f));
428 model = glm::translate(model, glm::vec3(0.0f, -0.65f, -0.12f));
429 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
430 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
431 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
432 //se programe cambio entre proyección ortogonal y perspectiva
433 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
434 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
435 color = glm::vec3(1.0f, 0.0f, 0.0f);
436 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
437 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
438

```

Ilustración 5. Figura roja 3.

```

438
439 //MEDIO DERECHA
440 model = glm::mat4(1.0);
441 //Traslación inicial para posicionar en -Z a los objetos
442 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
443 //otras transformaciones para el objeto
444 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
445 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
446 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
447 model = glm::translate(model, glm::vec3(-0.28f, 0.76f, 0.32f));
448 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
449 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
450 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
451 //se programe cambio entre proyección ortogonal y perspectiva
452 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
453 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
454 color = glm::vec3(1.0f, 0.0f, 0.0f);
455 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
456 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
457

```

Ilustración 6. Figura roja 4.

```

458
459 //ABAJO IZQUIERDA
460 model = glm::mat4(1.0);
461 //Traslación inicial para posicionar en -Z a los objetos
462 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
463 //otras transformaciones para el objeto
464 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
465 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
466 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
467 model = glm::translate(model, glm::vec3(0.55f, 0.26f, 0.15f));
468 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
469 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
470 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
471 //se programe cambio entre proyección ortogonal y perspectiva
472 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
473 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
474 color = glm::vec3(1.0f, 0.0f, 0.0f);
475 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
476 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
477

```

Ilustración 7. Figura roja 5 Incluye rotación para invertir y rotar la figura.

```

477
478 //MEDIO ABAJO
479 model = glm::mat4(1.0);
480 //Traslación inicial para posicionar en -Z a los objetos
481 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
482 //otras transformaciones para el objeto
483 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
484 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
485 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
486 model = glm::translate(model, glm::vec3(0.0f, 0.26f, 0.15f));
487 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
488 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
489 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
490 //se programe cambio entre proyección ortogonal y perspectiva
491 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
492 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
493 color = glm::vec3(1.0f, 0.0f, 0.0f);
494 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
495 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
496

```

Ilustración 8. Figura roja 6.

```

497
498 //ABAJO IZQUIERDA
499 model = glm::mat4(1.0);
500 //Traslación inicial para posicionar en -Z a los objetos
501 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
502 //otras transformaciones para el objeto
503 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
504 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
505 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
506 model = glm::translate(model, glm::vec3(-0.55f, 0.26f, 0.15f));
507 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
508 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
509 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
510 //se programe cambio entre proyección ortogonal y perspectiva
511 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
512 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
513 color = glm::vec3(1.0f, 0.0f, 0.0f);
514 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
515 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular

```

Ilustración 9. Figura roja 7.


```

516
517 //ABAJO DERECHA INVERSA
518 model = glm::mat4(1.0);
519 //Traslación inicial para posicionar en -Z a los objetos
520 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
521 //otras transformaciones para el objeto
522 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
523 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
524 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
525 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
526 model = glm::rotate(model, -35 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
527 //model = glm::rotate(model, 10 * toRadians, glm::vec3(0.0f, 0.0f, 0.0f));
528 model = glm::translate(model, glm::vec3(0.28f, -0.17f, 0.0f));
529 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
530 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
531 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
532 //se programe cambio entre proyección ortogonal y perspectiva
533 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
534 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
535 color = glm::vec3(1.0f, 0.0f, 0.0f);
536 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
537 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
538

```

Ilustración 10. Figura roja 8 (Incluye rotación para invertir y rotar la figura).

```

538
539 //ABAJO IZQUIERDA INVERSA
540 model = glm::mat4(1.0);
541 //Traslación inicial para posicionar en -Z a los objetos
542 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
543 //otras transformaciones para el objeto
544 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
545 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
546 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
547 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
548 model = glm::rotate(model, -35 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
549 //model = glm::rotate(model, 10 * toRadians, glm::vec3(0.0f, 0.0f, 0.0f));
550 model = glm::translate(model, glm::vec3(-0.28f, -0.17f, 0.0f));
551 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
552 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
553 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
554 //se programe cambio entre proyección ortogonal y perspectiva
555 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
556 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
557 color = glm::vec3(1.0f, 0.0f, 0.0f);
558 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
559 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
560
561

```

Ilustración 11. Figura roja 9 (Incluye rotación para invertir y rotar la figura).

RESULTADO:

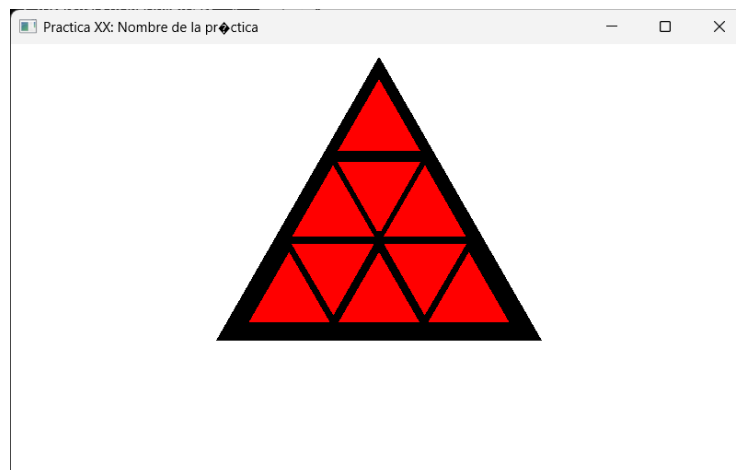


Ilustración 12. Resultado de cara roja.

Cara Azul:

Esta cara en general fue muy complicada debido a que debía de aplicarse rotaciones y traslaciones en los tres ejes, esto porque estaba en un ángulo específico que necesitábamos comprender para representar.

```
952 //////////////////////////////////////////////////
953 // STICKERS AZULES //
954 //////////////////////////////////////////////////
955
956 //PICO
957 model = glm::mat4(1.0);
958 //Traslación inicial para posicionar en -Z a los objetos
959 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
960 //otras transformaciones para el objeto
961
962 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
963 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
964 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
965 model = glm::translate(model, glm::vec3(0.05f, 1.25f, 0.58f));
966 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
967 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
968 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
969 //se programe cambio entre proyección ortogonal y perspectiva
970 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
971 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
972 color = glm::vec3(0.0f, 1.0f, 1.0f);
973 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
974 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
975
```

Ilustración 13. Figura Azul 1.

```
976 //MEDIO IZQUIERDO
977 model = glm::mat4(1.0);
978 //Traslación inicial para posicionar en -Z a los objetos
979 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
980 //otras transformaciones para el objeto
981
982 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
983 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
984 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
985 model = glm::translate(model, glm::vec3(0.35f, 0.76f, 0.45f));
986 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
987 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
988 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
989 //se programe cambio entre proyección ortogonal y perspectiva
990 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
991 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
992 color = glm::vec3(0.0f, 1.0f, 1.0f);
993 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
994 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
995
```

Ilustración 14. Figura Azul 2.

```

995
996 //MEDIO MEDIO
997 model = glm::mat4(1.0);
998 //Traslación inicial para posicionar en -Z a los objetos
999 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1000 //otras transformaciones para el objeto
1001
1002 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1003 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la
1004 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1005 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
1006 model = glm::rotate(model, -45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
1007 model = glm::rotate(model, 160 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1008
1009 //model = glm::translate(model, glm::vec3(0.7f, 0.18f, 0.15f)); // goat
1010 model = glm::translate(model, glm::vec3(0.875f, -0.3f, -0.01f)); // goat
1011
1012 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota = -60 y x rota = 180
1013 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1014 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1015 color = glm::vec3(0.0f, 1.0f, 1.0f);
1016 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1017 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1018

```

Ilustración 15. Figura Azul 3.

```

1019 //MEDIO DERECHO
1020 model = glm::mat4(1.0);
1021 //Traslación inicial para posicionar en -Z a los objetos
1022 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1023 //otras transformaciones para el objeto
1024
1025 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1026 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1027 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1028 model = glm::translate(model, glm::vec3(0.1f, 0.76f, 0.9f));
1029 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1030 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1031 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
1032 //se programe cambio entre proyección ortogonal y perspectiva
1033 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
1034 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
1035 color = glm::vec3(0.0f, 1.0f, 1.0f);
1036 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1037 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1038

```

Ilustración 16. Figura Azul 4.

```

1039 //ABAJO IZQUIERDA
1040 model = glm::mat4(1.0);
1041 //Traslación inicial para posicionar en -Z a los objetos
1042 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1043 //otras transformaciones para el objeto
1044
1045 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1046 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1047 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1048 model = glm::translate(model, glm::vec3(0.6f, 0.3f, 0.32f));
1049 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1050 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1051 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
1052 //se programe cambio entre proyección ortogonal y perspectiva
1053 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
1054 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
1055 color = glm::vec3(0.0f, 1.0f, 1.0f);
1056 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1057 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1058

```

Ilustración 17. Figura Azul 5.


```

1058
1059 //ABAJO DERECHA INVERSO
1060 model = glm::mat4(1.0);
1061 //Traslación inicial para posicionar en -Z a los objetos
1062 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1063 //otras transformaciones para el objeto
1064
1065 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1066 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1067 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1068 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
1069 model = glm::rotate(model, -45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
1070 model = glm::rotate(model, 160 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1071
1072 model = glm::translate(model, glm::vec3(0.7f, 0.18f, 0.15f)); // goat
1073
1074 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota = -
1075 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1076 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1077 color = glm::vec3(0.0f, 1.0f, 1.0f);
1078 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1079 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1080

```

Ilustración 18. Figura Azul 6.

```

1081 //ABAJO MEDIO
1082 model = glm::mat4(1.0);
1083 //Traslación inicial para posicionar en -Z a los objetos
1084 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1085 //otras transformaciones para el objeto
1086
1087 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1088 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1089 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1090 model = glm::translate(model, glm::vec3(0.35f, 0.3f, 0.75f));
1091 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1092 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1093 //La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
1094 //se programe cambio entre proyección ortogonal y perspectiva
1095 //glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
1096 //glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
1097 color = glm::vec3(0.0f, 1.0f, 1.0f);
1098 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1099 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1100

```

Ilustración 19. Figura Azul 7.

```

1101 //ABAJO DERECHA
1102 model = glm::mat4(1.0);
1103 //Traslación inicial para posicionar en -Z a los objetos
1104 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1105 //otras transformaciones para el objeto
1106
1107 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1108 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1109 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1110 model = glm::translate(model, glm::vec3(0.08f, 0.3f, 1.2f));
1111 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1112 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1113 //La línea de proyección solo se manda una vez a menos que en tiempo de ejecución
1114 //se programe cambio entre proyección ortogonal y perspectiva
1115 //glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
1116 //glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
1117 color = glm::vec3(0.0f, 1.0f, 1.0f);
1118 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1119 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1120

```

Ilustración 20. Figura Azul 8.

```

1121 //ABAJO IZQUIERDA INVERSO
1122 model = glm::mat4(1.0);
1123 //Traslación inicial para posicionar en -Z a los objetos
1124 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
1125 //otras transformaciones para el objeto
1126
1127 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
1128 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
1129 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
1130 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
1131 model = glm::rotate(model, -45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
1132 model = glm::rotate(model, 160 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
1133
1134 //model = glm::translate(model, glm::vec3(0.7f, 0.18f, 0.15f)); // goat
1135 model = glm::translate(model, glm::vec3(0.875f, 0.18f, -0.32f)); // goat
1136
1137 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota =
1138 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
1139 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
1140 color = glm::vec3(0.0f, 1.0f, 1.0f);
1141 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
1142 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
1143

```

Ilustración 21. Figura Azul 9.

RESULTADO:

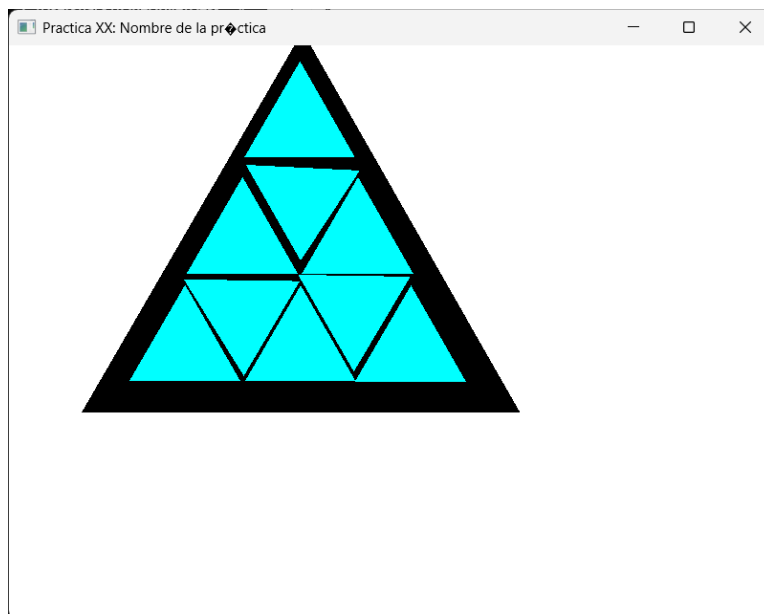


Ilustración 22. Resultado cara azul.

CARA AMARILLA:

Esta cara es el opuesto de la cara azul, tomando ángulos similares, pero en signo contrario en x y z al menos en las rotaciones para las inversiones de las figuras. Esta figura igual fue un problema por combinar las rotaciones de los ángulos.

```
752 //////////////////////////////////////////////////
753 // STICKERS AMARILLOS //
754 //////////////////////////////////////////////////
755 //PICO
756 model = glm::mat4(1.0);
757 //Traslación inicial para posicionar en -Z a los objetos
758 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
759 //otras transformaciones para el objeto
760
761 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
762 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
763 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
764 model = glm::translate(model, glm::vec3(-0.05f, 1.25f, 0.58f));
765 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
766 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
767 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
768 //se programe cambio entre proyección ortogonal y perspectiva
769 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
770 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
771 color = glm::vec3(1.0f, 1.0f, 0.0f);
772 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
773 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
774
```

Ilustración 23. Figura Amarilla 1.

```
776 //MEDIO IZQUIERDO
777 model = glm::mat4(1.0);
778 //Traslación inicial para posicionar en -Z a los objetos
779 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
780 //otras transformaciones para el objeto
781
782 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
783 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
784 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
785 model = glm::translate(model, glm::vec3(-0.35f, 0.76f, 0.45f));
786 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
787 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
788 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
789 //se programe cambio entre proyección ortogonal y perspectiva
790 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
791 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
792 color = glm::vec3(1.0f, 1.0f, 0.0f);
793 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
794 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
795
```

Ilustración 24. Figura Amarilla 2.

```

796 //MEDIO DERECHO
797 model = glm::mat4(1.0);
798 //Traslación inicial para posicionar en -Z a los objetos
799 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
800 //otras transformaciones para el objeto
801
802 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
803 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
804 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
805 model = glm::translate(model, glm::vec3(-0.05f, 0.76f, 0.9f));
806 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
807 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
808 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
809 //se programe cambio entre proyección ortogonal y perspectiva
810 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
811 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
812 color = glm::vec3(1.0f, 1.0f, 0.0f);
813 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
814 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
815

```

Ilustración 25. Figura Amarilla 3.

```

816 //MEDIO MEDIO
817 model = glm::mat4(1.0);
818 //Traslación inicial para posicionar en -Z a los objetos
819 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
820 //otras transformaciones para el objeto
821
822 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
823 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la t
824 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
825 model = glm::rotate(model, -53 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
826 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
827 model = glm::rotate(model, 161 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
828
829 model = glm::translate(model, glm::vec3(-0.925f, -0.18f, 0.05f)); // goat
830
831 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota = -60 y x rota = 180
832 //model = glm::translate(model, glm::vec3(-0.45f, -0.85f, -0.45f)); // posición en la cara amarilla
833 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
834 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
835 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
836 //se programe cambio entre proyección ortogonal y perspectiva
837 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
838 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
839 color = glm::vec3(1.0f, 1.0f, 0.0f);
840 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
841 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
842

```

Ilustración 26. Figura Amarilla 4.

```

845 //ABAJO IZQUIERDA
846 model = glm::mat4(1.0);
847 //Traslación inicial para posicionar en -Z a los objetos
848 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
849 //otras transformaciones para el objeto
850
851
852 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
853 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
854 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
855 model = glm::translate(model, glm::vec3(-0.6f, 0.3f, 0.28f));
856 model = glm::scale(model, glm::vec3(0.6f, 0.5f, 0.5f));
857 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
858 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
859 //se programe cambio entre proyección ortogonal y perspectiva
860 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
861 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
862 color = glm::vec3(1.0f, 1.0f, 0.0f);
863 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
864 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
865

```

Ilustración 27. Figura Amarilla 5.

```

866 //ABAJO IZQUIERDA INVERSO
867 model = glm::mat4(1.0);
868 //Traslación inicial para posicionar en -Z a los objetos
869 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
870 //otras transformaciones para el objeto
871
872 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
873 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la t
874 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
875 model = glm::rotate(model, -53 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
876 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
877 model = glm::rotate(model, 161 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
878
879 model = glm::translate(model, glm::vec3(-0.7f, 0.3f, 0.15f)); // goat
880
881 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota = -60 y x rota = 180
882 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
883 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
884 color = glm::vec3(1.0f, 1.0f, 0.0f);
885 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
886 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
887

```

Ilustración 28. Figura Amarilla 6.

```

887
888 //ABAJO MEDIO
889 model = glm::mat4(1.0);
890 //Traslación inicial para posicionar en -Z a los objetos
891 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
892 //otras transformaciones para el objeto
893
894 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
895 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
896 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
897 model = glm::translate(model, glm::vec3(-0.35f, 0.3f, 0.8f));
898 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
899 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
900 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
901 //se programe cambio entre proyección ortogonal y perspectiva
902 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
903 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
904 color = glm::vec3(1.0f, 1.0f, 0.0f);
905 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
906 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
907

```

Ilustración 29. Figura Amarilla 7.

```

908 //ABAJO DERECHA
909 model = glm::mat4(1.0);
910 //Traslación inicial para posicionar en -Z a los objetos
911 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
912 //otras transformaciones para el objeto
913
914 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
915 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
916 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
917 model = glm::translate(model, glm::vec3(-0.08f, 0.3f, 1.25f));
918 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
919 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
920 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
921 //se programe cambio entre proyección ortogonal y perspectiva
922 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
923 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
924 color = glm::vec3(1.0f, 1.0f, 0.0f);
925 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
926 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
927

```

Ilustración 30. Figura Amarilla 8.


```

928 //ABAJO DERECHA INVERSO
929 model = glm::mat4(1.0);
930 //Traslación inicial para posicionar en -Z a los objetos
931 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
932 //otras transformaciones para el objeto
933
934 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
935 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f)); //al presionar la t
936 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
937 model = glm::rotate(model, -53 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
938 model = glm::rotate(model, 45 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
939 model = glm::rotate(model, 161 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
940
941 //model = glm::translate(model, glm::vec3(-1.0f, 0.15f, -0.1f)); // goat
942 model = glm::translate(model, glm::vec3(-0.9f, 0.23f, -0.34f)); // goat y debe de ser cercana a 0
943
944 //model = glm::translate(model, glm::vec3(0.425f, -0.7f, -0.5f)); // posición cuando y rota = -60 y x rota = 180
945 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
946 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
947 color = glm::vec3(1.0f, 1.0f, 0.0f);
948 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
949 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
950
951

```

Ilustración 31. Figura Amarilla 9.

RESULTADO:

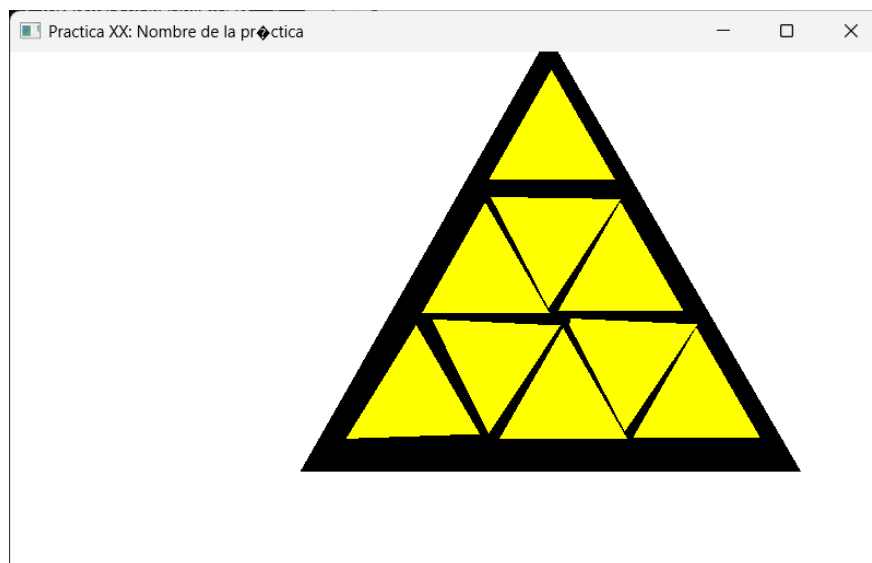


Ilustración 32. Resultado final Cara Amarilla.

CARA VERDE:

La cara verde representa la cara inferior de la pirámide, por lo que es fácil guiarse con el eje X y Y, permitiendo determinar las pirámides “sticker” que conforman la pirámide principal.

```
561
562 //////////////////////////////////////////////////
563 //STICKERS VERDES//
564 //////////////////////////////////////////////////
565
566 //ABAJO MEDIO
567 model = glm::mat4(1.0);
568 //Traslación inicial para posicionar en -Z a los objetos
569 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
570 //otras transformaciones para el objeto
571
572 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
573 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
574 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
575
576 model = glm::translate(model, glm::vec3(0.0f, 0.13f, 0.22f));
577 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
578 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
579 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
580 //se programe cambio entre proyección ortogonal y perspectiva
581 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
582 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
583 color = glm::vec3(0.0f, 1.0f, 0.0f);
584 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
585 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
586
```

Ilustración 33. Figura verde 1.

```
587 //ABAJO IZQUIERDA
588 model = glm::mat4(1.0);
589 //Traslación inicial para posicionar en -Z a los objetos
590 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
591 //otras transformaciones para el objeto
592
593 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
594 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
595 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
596
597 model = glm::translate(model, glm::vec3(-0.55f, 0.13f, 0.22f));
598 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
599 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
600 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
601 //se programe cambio entre proyección ortogonal y perspectiva
602 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
603 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
604 color = glm::vec3(0.0f, 1.0f, 0.0f);
605 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
606 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
607
```

Ilustración 34. Figura verde 2.

```

608 //ABAJO IZQUIERDA INVERSA
609 model = glm::mat4(1.0);
610 //Traslación inicial para posicionar en -Z a los objetos
611 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
612 //otras transformaciones para el objeto
613
614 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
615 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
616 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
617 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
618 model = glm::translate(model, glm::vec3(-0.28f, 0.13f, -0.28f));
619 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
620 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
621 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
622 //se programe cambio entre proyección ortogonal y perspectiva
623 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
624 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
625 color = glm::vec3(0.0f, 1.0f, 0.0f);
626 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
627 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
628

```

Ilustración 35. Figura verde 3.

```

629 //ABAJO DERECHA
630 model = glm::mat4(1.0);
631 //Traslación inicial para posicionar en -Z a los objetos
632 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
633 //otras transformaciones para el objeto
634
635 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
636 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
637 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
638
639 model = glm::translate(model, glm::vec3(0.55f, 0.13f, 0.22f));
640 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
641 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
642 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
643 //se programe cambio entre proyección ortogonal y perspectiva
644 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
645 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
646 color = glm::vec3(0.0f, 1.0f, 0.0f);
647 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
648 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
649

```

Ilustración 36. Figura verde 4.

```

650 //ABAJO DERECHA INVERSA
651 model = glm::mat4(1.0);
652 //Traslación inicial para posicionar en -Z a los objetos
653 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
654 //otras transformaciones para el objeto
655
656 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
657 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
658 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
659 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
660 model = glm::translate(model, glm::vec3(0.28f, 0.13f, -0.28f));
661 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
662 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
663 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
664 //se programe cambio entre proyección ortogonal y perspectiva
665 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
666 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
667 color = glm::vec3(0.0f, 1.0f, 0.0f);
668 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
669 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
670

```

Ilustración 37. Figura verde 5

```

671 //MEDIO IZQUIERDA
672 model = glm::mat4(1.0);
673 //Traslación inicial para posicionar en -Z a los objetos
674 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
675 //otras transformaciones para el objeto
676
677 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
678 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
679 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
680 model = glm::translate(model, glm::vec3(-0.28f, 0.13f, 0.76f));
681 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
682 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
683 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
684 //se programe cambio entre proyección ortogonal y perspectiva
685 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
686 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
687 color = glm::vec3(0.0f, 1.0f, 0.0f);
688 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
689 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
690

```

Ilustración 38. Figura verde 6.

```

691 //MEDIO MEDIO
692 model = glm::mat4(1.0);
693 //Traslación inicial para posicionar en -Z a los objetos
694 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
695 //otras transformaciones para el objeto
696
697 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
698 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
699 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
700 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
701 model = glm::translate(model, glm::vec3(0.0f, 0.13f, -0.77f));
702 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
703 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
704 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
705 //se programe cambio entre proyección ortogonal y perspectiva
706 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
707 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
708 color = glm::vec3(0.0f, 1.0f, 0.0f);
709 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
710 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
711

```

Ilustración 39. Figura verde 7.

```

712 //MEDIO DERECHA
713 model = glm::mat4(1.0);
714 //Traslación inicial para posicionar en -Z a los objetos
715 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
716 //otras transformaciones para el objeto
717
718 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
719 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
720 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
721 model = glm::translate(model, glm::vec3(0.28f, 0.13f, 0.76f));
722 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
723 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
724 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
725 //se programe cambio entre proyección ortogonal y perspectiva
726 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
727 glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
728 color = glm::vec3(0.0f, 1.0f, 0.0f);
729 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
730 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
731

```

Ilustración 40. Figura verde 8.

```

732 //PICO
733 model = glm::mat4(1.0);
734 //Traslación inicial para posicionar en -Z a los objetos
735 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -3.0f));
736 //otras transformaciones para el objeto
737
738 model = glm::rotate(model, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
739 model = glm::rotate(model, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
740 model = glm::rotate(model, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));
741 model = glm::translate(model, glm::vec3(0.0f, 0.13f, 1.25f));
742 model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
743 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
744 //la línea de proyección solo se manda una vez a menos que en tiempo de ejecución
745 //se programe cambio entre proyección ortogonal y perspectiva
746 //glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
747 //glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
748 color = glm::vec3(0.0f, 1.0f, 0.0f);
749 glUniform3fv(uniformColor, 1, glm::value_ptr(color)); //para cambiar el color del objetos
750 meshList[1]->RenderMesh(); //dibuja cubo y pirámide triangular
751

```

Ilustración 41. Figura verde 9.

RESULTADO:

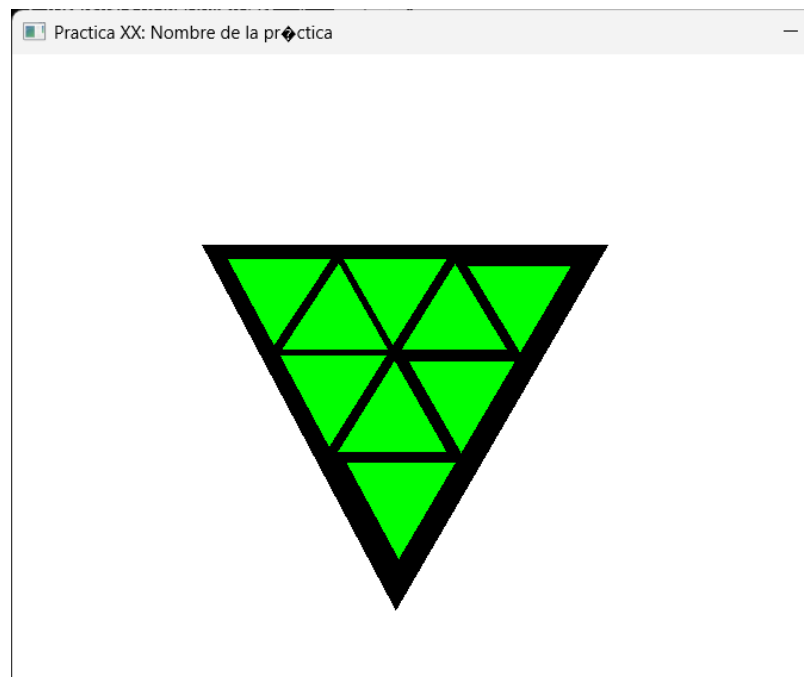


Ilustración 42. Resultado final Cara verde.

RESULTADO FINAL:

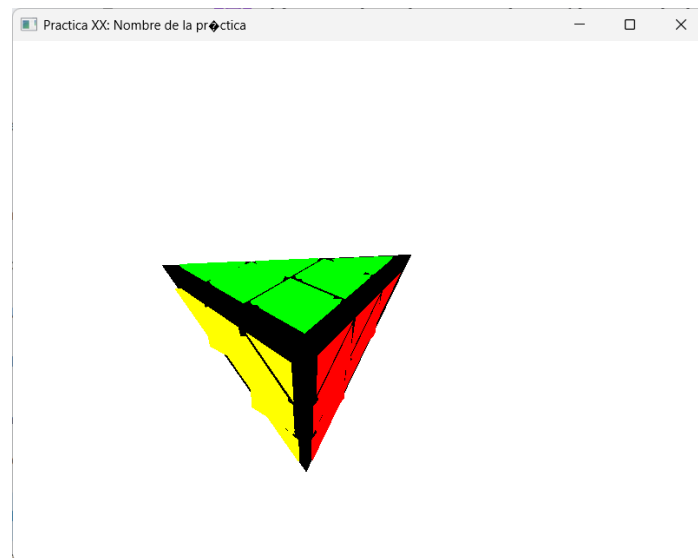


Ilustración 43. Figura mostrando 3 caras.

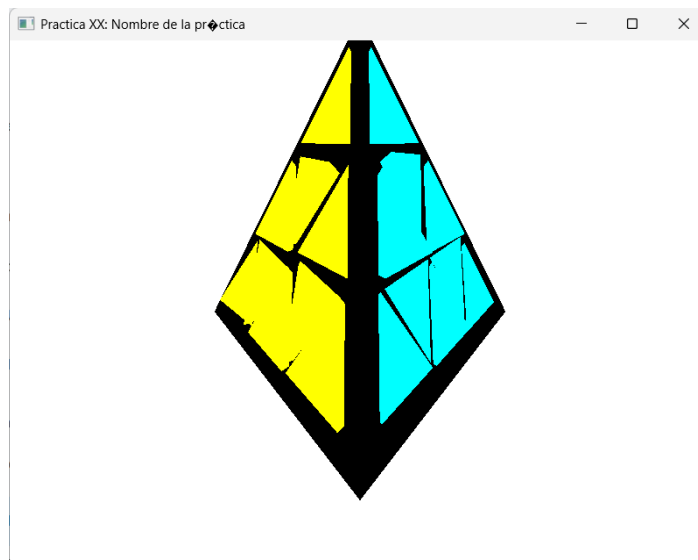


Ilustración 44. Figura mostrando dos caras diagonales.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

- Rotaciones y sentido: El principal error de mi figura, sobre todo para la cara azul y amarillo, el problema consistió en tener que hacer rotaciones, pues estas rotaciones cambian el sentido de los ejes. Por lo que la ubicación por medio de coordenadas se dificultó. Además, que los ángulos de un eje interferían con los de otro eje. Por lo que el cálculo de los ángulos específicos fue una gran complicación. Incluso quedando expuesto en las mismas caras azules y amarillos, en los que la rotación no quedó 100% exacta.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

La complejidad del problema en sí fue normal, básicamente sólo fue ubicar pirámides sobre otra que actúa como base. Lo único que aumenta la dificultad es el tener que rotar figuras y tener en cuenta los ángulos y cómo las rotaciones cambian las direcciones de los ejes.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

La explicación de la práctica fue muy buena, en base a lo revisado en el laboratorio fue que se realizaron las traslaciones correspondientes. Se nos advirtió de las dificultades que podría haber. Por lo que al tener esas consideraciones fue mucho más fácil la resolución de la práctica.

c. Conclusión

El uso de las dimensiones, traslaciones y rotaciones permitieron ver la creación de figuras y el juego que se hace en las 3 dimensiones. Permitiendo comprender a fondo como funcionan las figuras y sus propiedades. Al final, la práctica es el inicio del entendimiento del modelado en 3D que nos ayudará a realizar figuras mucho más complicadas a futuro. En general la práctica actúa como fundamentos y base de las siguientes.

1. Bibliografía en formato APA